A New Algorithm for the Assignment Problem

Technical presentation based on Bertsekas (1981)

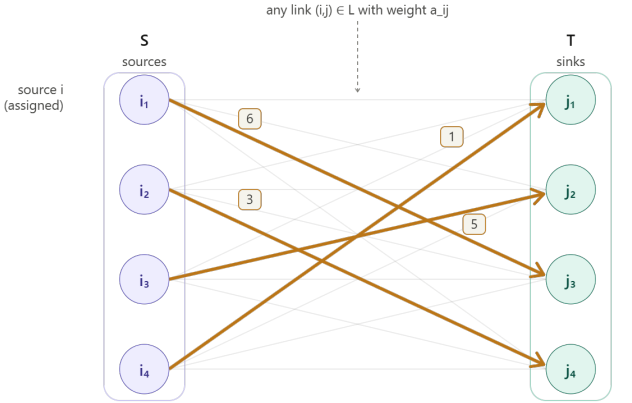
SL224 Network Science and Optimisation
Kartikeya Gaur
26371

April 19, 2026

Paper at a glance

- **Source:** D. P. Bertsekas, *A New Algorithm for the Assignment Problem*, *Mathematical Programming*, 1981.
- Problem: maximum-weight bipartite matching / assignment on an $N \times N$ matrix $A = (a_{ij})$.
- Main contribution: the **auction algorithm**, a primal-dual method related to the Hungarian method but with much more aggressive price updates.
- Key empirical claim: for randomly generated dense problems N being several hundreds our method can converge $10\times$ or more than the Hungarian method.

Assignment problem



Assumptions

Throughout the paper they have assumed the following:

(a) There exists at least one assignment of cardinality N. [termination criteria]

(b) The weights a_{ij} are nonnegative integers not all zero and the maximal weight will be denoted by R, i.e.,

$$R = \max_{(i,j)\in L} a_{ij} > 0.$$

(c) For each source i there are at least two links (i, j) in L. [reducing dimensionality]

Assignment problem formulation : Primal

Let S be sources and T be sinks with $|S| = |T| = N$.

Primal problem

$$\max \sum_{(i,j)\in L} a_{ij} x_{ij}$$

subject to

$$\sum_{j:(i,j)\in L} x_{ij} = 1 \quad (i \in S), \qquad \sum_{i:(i,j)\in L} x_{ij} = 1 \quad (j \in T), \qquad x_{ij} \in \{0, 1\}.$$

- For the full dense case, $L = S \times T$.
- The solution is a perfect matching maximizing total value.

Assignment problem formulation : Primal

Let S be sources and T be sinks with $|S| = |T| = N$.

Primal problem

$$\max \sum_{(i,j)\in L} a_{ij} x_{ij}$$

subject to

$$\sum_{j:(i,j)\in L} x_{ij} = 1 \quad (i \in S), \qquad \sum_{i:(i,j)\in L} x_{ij} = 1 \quad (j \in T), \qquad x_{ij} \geq 0.$$

- The constraints are simply saying that the argument matrix X must be a permutation matrix. Since the constraint matrices are totally **unimodular**, the integer programming problem is equivalent to its linear programming relaxation.

Assignment problem formulation : Dual

The corresponding dual problem in the vectors $m = (m_1, \ldots, m_N)$, $p = (p_1, \ldots, p_N)$ is

$$\begin{aligned} \text{minimize} \quad & \sum_{i=1}^{N} m_i + \sum_{j=1}^{N} p_j, \\ \text{subject to} \quad & m_i + p_j \geq a_{ij}, \quad \forall (i,j) \in L. \end{aligned} \tag{1}$$

Interpretation for auction viewpoint

p_j is the **price** of sink j and m_i is the best net profit currently available to source i.

- If feasibility and complementary slackness hold and $|X| = N$, primal-dual optimality follows immediately.
- The algorithm therefore focuses on preserving these relations while driving the matching to cardinality N.

Assignment problem formulation : Dual

The *complementary slackness* is

$$x_{ij}(a_{ij} - m_i - p_j) = 0, \quad \forall (i,j) \in L. \tag{2}$$

From complementary slackness we have that if (i,j) is part of an optimal assignment, then for any optimal solution (m,p) of the dual problem we have

$$m_i = a_{ij} - p_j = \max\{a_{in} - p_n \mid \text{all } n \text{ with } (i,n) \in L\}, \tag{3}$$

i.e., at an optimum source i is assigned to a sink j offering maximum profit margin relative to the price vector p.

Auction POV

For each source i, define the current best profit margin

$$\tilde{m}_i = \max_{j:(i,j)\in L} \{a_{ij} - p_j\}.$$

Choose a best sink

$$\bar{j} \in \arg\max_j (a_{ij} - p_j),$$

and the second-best margin

$$\tilde{m}_i = \max_{j \neq \bar{j}} (a_{ij} - p_j).$$



Bidding intuition

The source bids for its best sink and raises that sink's price just enough to match the runner-up option.

This "maximal safe bid" drives **outpricing**.

Iteration 1: Initialization and Case 1

The algorithm starts with an empty assignment X^0. Initial dual variables are set to $p_j^0 = 0$ and $m_i^0 = 10$ to ensure feasibility.

Initial Tableau

	p_j	0	0	0	0
j	m_i				
	10	1	3	6	1
	10	2	4	7	3
	10	2	5	7	2
	10	1	3	5	1

- **Calculations:** $\bar{m} = \max\{a_{2j} - p_j\} = 6$ (at $\bar{j} = 3$) and $\tilde{m} = 3$.
- **Step Taken:** Since $\bar{m} > \tilde{m}$, the algorithm enters **Case 1**.
- **Result:** p_3 is increased by $\bar{m} - \tilde{m} = 3$, and source 2 is assigned to sink 3.

Iteration 4: Case 2 and Dual Variable Change

j	m_i \ p_j	0	0	3	0
3	3	1	3	6	1
	10	2	4	7	3
	10	2	5	7	2
	10	1	3	5	1

After 1st iteration

j	m_i \ p_j	0	0	3	0
3	3	1	3	6	1
2	4	2	4	7	3
	10	2	5	7	2
	10	1	3	5	1

After 2nd iteration

j	m_i \ p_j	0	1	3	0
3	3	1	3	6	1
	4	2	4	7	3
2	4	2	5	7	2
	10	1	3	5	1

After 3rd iteration

j	m_i \ p_j	0	2	4	0
3	2	1	3	6	1
	4	2	4	7	3
	4	2	5	7	2
2	1	1	3	5	1

After 4th iteration

Iteration 6: Final Augmentation and Termination

j	m_i	p_j	0	2	4	0
3	2		1	3	6	1
4	3		2	4	7	3
	4		2	5	7	2
2	1		1	3	5	1

After 5th iteration

j	m_i	p_j	0	2	4	0
3	2		1	3	6	1
4	3		2	4	7	3
2	3		2	5	7	2
1	1		1	3	5	1

After 6th iteration

- The algorithm terminates with an optimal assignment of cardinality $N = 4$.

Why it can beat the Hungarian method

Core phenomenon: outpricing

- A sink is *outpriced* once its price is too high to remain competitive for many sources.
- After that point, later scans largely ignore it, shrinking the effective search space.

Structural difference

Auction uses fewer, larger price jumps; Hungarian typically uses smaller, more incremental updates.

- The Hungarian method focuses on assigning unassigned sources by searching directly for augmenting paths along links (i, j) where $m_i + p_j = a_{ij}$. Then dual variables changes.
- In the proposed method, the roles of searching for an augmenting path and adjusting dual variables are effectively **reversed**.

Two algorithms with comparable worst case complexity can differ substantially in their performance in sovling randomly generated problems or problems typically arising in practice.

Worst-case complexity

For full dense integer-valued problems with entries in $[0, R]$:

- Iterations that augment the matching occur at most N times.
- Non-augmenting iterations raise one or more prices by at least one integer unit.
- Using bounded initial prices, Bertsekas derives

$$O(N^3) + O(RN^2)$$

for the **pure** auction algorithm.

Implication

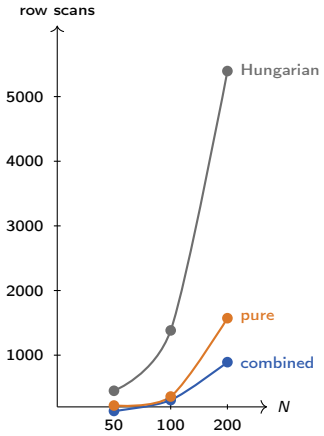
The pure method can be worse than Hungarian in worst-case asymptotics when R is large, even though it is often faster in practice.

Computational results reported in the paper

N	$R = 100$			$R = 100,000$		
	Hungarian	combine	Pure	Hungarian	combine	Pure
100	0.319	0.103	0.103	0.459	0.13	0.3
400	12.5	2.4	2.4	24.5	3.64	12.9

Distribution	Hungarian	Combined	Pure
Uniform	24.5	3.64	12.9
Normal	24.8	2.45	9.55

Table: Comparison of execution times (seconds) for $N = 400$ and $R = 100,000$ at maximum weight range.



- Values vary with $R \in \{30, 100, 10{,}000\}$.

Takeaways

1. The auction algorithm is a primal-dual assignment method with an intuitive bidding interpretation.
2. Its strength is **outpricing**: large admissible price increases remove sinks from future contention.
3. Pure-form complexity is $O(N^3) + O(RN^2)$; a hybrid with Hungarian search restores $O(N^3)$ worst-case behavior.
4. Experiments show strong speedups on dense random instances, especially as N grows.

Questions?

Primal-Dual Proof

Proof.

Let $\{p_j\}$ be any specification of prices. We have

$$A_{ik_i} \leq (A_{ik_i} - p_{k_i}) + p_{k_i} \leq \max_j\{A_{ij} - p_j\} + p_{k_i}.$$

Hence the total payoff for the assignment $\{(i, k_i)\}$ satisfies

$$\sum_{i=1}^{n} A_{ik_i} \leq \sum_{i=1}^{n} \max_j\{A_{ij} - p_j\} + \sum_{j=1}^{n} p_j.$$

That is, any value of the objective function of primal is at least as large as the optimal value of dual. Then

$$\sum_{i=1}^{n} A_{ik_i} = \sum_{i=1}^{n}(A_{ik_i} - p_{k_i}) + \sum_{j=1}^{n} p_j = \sum_{i=1}^{n} \max_j\{A_{ij} - p_j\} + \sum_{j=1}^{n} p_j.$$

It then follows that optimal values of the two problems are equal. □

Termination

Proposition 1. *The algorithm terminates at an optimal assignment in finitely many iterations.*

- Assume, for contradiction, that the algorithm never terminates.
- After some point, assignment cardinality stays fixed, while at each later iteration at least one m_i decreases and at least one p_j increases.
- Define
$$S_\infty = \{i \in S \mid m_i^k \to -\infty\}, \qquad T_\infty = \{j \in T \mid p_j^k \to \infty\}.$$
- Both sets are nonempty; for large k, every sink in T_∞ is assigned to a source in S_∞.
- Since the algorithm still has not terminated, some source in S_∞ remains unassigned, so $|S_\infty| > |T_\infty|$.
- Feasibility implies neighbors of sources in S_∞ must lie inside T_∞, contradicting the existence of a full assignment.

Hence the algorithm must terminate; combined with primal-dual feasibility, the final assignment is optimal.

Example: lower-triangular matrix family

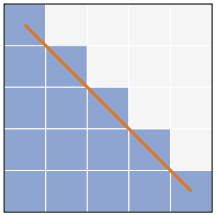
Consider the dense matrix

$$a_{ij} = \begin{cases} N + 1 - j, & j \le i, \\ 0, & j > i. \end{cases}$$

Starting from $p = 0$ and no assignments:

- Iteration 1 assigns $(1, 1)$ and raises p_1 by N.
- Iteration 2 assigns $(2, 2)$ and raises p_2 by $N - 1$.
- Iteration k assigns (k, k) and outprices sink k.

dense lower triangle



diagonal assignments become successively outpriced

Complexity on this family

Auction: $O(N^2)$ on this instance family; Hungarian: $O(N^3)$.

Combined auction + Hungarian strategy

To recover an $O(N^3)$ worst-case guarantee, the paper proposes switching to Hungarian search when auction progress stalls or empirically R>100.

- Maintain two queues of unassigned sources.
- A **cycle** ends when the first queue is exhausted.
- If a cycle makes no augmentation, or if expensive Case 2 updates happen too often, increment a counter.
- Switch to the Hungarian method once the counter exceeds a threshold (set to $0.1N$ in the experiments).

Practical takeaway

Use auction dynamics to exploit fast early and middle progress; use Hungarian search to clean up the hard tail cases.

References

1. Bertsekas, D. P., *A New Algorithm for the Assignment Problem*, *Mathematical Programming*, 1981.
   `https://web.eecs.umich.edu/~pettie/matching/Bertsekas-a-new-algorithm-assignment-problem-Mathematical-Programming-1981.pdf`
2. Ahuja, Ravindra K., Magnanti, Thomas L., and Orlin, James B., *Network Flows: Theory, Algorithms, and Applications*.
3. Jin, project report, Stanford CME 323.
   `https://stanford.edu/~rezab/classes/cme323/S16/projects_reports/jin.pdf`
4. Illustration credit: freepik.com
5. AI tools used: Gemini, Claude and Prism (OpenAI).